

ONLINE SUPPLEMENT FOR

“SHOULD WE ALWAYS SCRAMBLE DIGITAL NETS FOR QUASI-MONTE CARLO?”

TO BE PUBLISHED IN THE PROCEEDINGS OF WSC’2026

Pierre L’Ecuyer¹ and Christiane Lemieux²,

¹DIRO, Université de Montréal, Montréal, QC, CANADA

²Department of Statistics and Actuarial Science, University of Waterloo, CANADA

ABSTRACT

The purpose of this supplement is to provide a proof for Proposition 2 and to report a more extensive subset of experimental results for our WSC’2026 paper.

1 PROOF OF PROPOSITION 2

Proposition 2. Let f be the two-dimensional function defined by $f(u_1, u_2) = u_1 u_2$ for $0 \leq u_1, u_2 \leq 1$. Let P_n be the $n = 2^k$ first points of a digital (0,2)-sequence in base 2 for which the first generating matrix $\mathbf{C}_1$ is the identity, $k \geq 6$, and let $\hat{\mu}_n$ be the estimator obtained from applying a RDS and the tent transformation to P_n . Then this f is integrated with zero error (no bias and zero variance) by $\hat{\mu}_n$.

Proof. The variance of the unbiased estimator $\hat{\mu}_n$ is equal to the variance of the estimator based on P_n with a RDS on the function $g(u_1, u_2) = f(\phi(u_1), \phi(u_2)) = (1 - |2u_1 - 1|)(1 - |2u_2 - 1|)$, which is given by $\sum_{\mathbf{0} \neq \mathbf{h} \in P_n^\top} |\tilde{g}(\mathbf{h})|^2$, where $P_n^\top$ is the dual space corresponding to P_n . It consists of vectors $\mathbf{h} = (h_1, h_2)$ such that $\mathbf{C}_1^\top \tilde{\mathbf{h}}_1 + \mathbf{C}_2^\top \tilde{\mathbf{h}}_2 = \mathbf{0}$, where $\tilde{\mathbf{h}} = (h_0, h_1, \dots, h_m, 0, \dots)$ is the binary representation of h . Here we assume $\mathbf{C}_1$ and $\mathbf{C}_2$ are the $\infty \times k$ generating matrices of the $(0, k, 2)$ -net. Then, only the first k bits of h_1 and h_2 matter to determine whether or not $\mathbf{h}$ is in the dual $P_n^\top$. The proof of the proposition follows by combining the following two lemmas. $\square$

Lemma 1. The Walsh coefficients $\tilde{g}(h)$ of $\phi(u) = (1 - |2u - 1|)$ are nonzero if and only if h is of the form $h = 1 + 2^a$ for $a \geq 1$, or if $h = 0$.

Proof. If $h = 0$, then $\tilde{g}(h) = \int_0^1 \phi(u) du = 1/2$. If $h = 1$, then $\tilde{g}(h) = \int_0^{1/2} 2u du - \int_{1/2}^1 2(1 - u) du = 0$.

For $h > 1$ (and therefore $m \geq 1$), let $z = (z_1, \dots, z_{2^{m+1}}) \in \{-1, 1\}^{2^{m+1}}$ be the sequence such that $(-1)^{h \cdot x}$ is equal to z_j when $x \in [(j-1)2^{-m-1}, j2^{-m-1})$. Note that z depends on h , and as such we may write $z(h)$ when appropriate. Note also that $\sum_{j=1}^{2^{m+1}} z_j = 0$. Using this notation, we write

$$\tilde{g}(h) = \sum_{j=1}^{2^{m+1}} z_j \phi_j \text{ where } \phi_j = \int_{(j-1)/2^{m+1}}^{j/2^{m+1}} g(u) du.$$

The symmetry of g implies that $\phi_j = \phi_{2^{m+1}-j+1}$ for $j = 1, \dots, 2^{m+1}$ and thus $\tilde{g}(h) = \sum_{j=1}^{2^m} (z_j + z_{2^{m+1}-j+1}) \phi_j$.

If h is such that $h_0 = 0$, then the string $z(h)$ is such that $z_j = z_{2^m+j}$ and thus

$$\tilde{g}(h) = \sum_{j=1}^{2^m} z_j (\phi_j + \phi_{2^m-j+1}) = \sum_{j=1}^{2^m} z_j \left(\frac{2j+1}{2^{m+1}} + \frac{2(2^m-j+1)+1}{2^{m+1}} \right) = \left((1 + 2^{-(m-1)}) \sum_{j=1}^{2^m} z_j \right) = 0.$$

If $h = 1 + 2^a$ for some $a \geq 1$, then $m = a$ and the corresponding string $z(h)$ is such that for any odd $j < 2^m$, $z_j z_{j+1} = [1, -1]$, i.e., its first half is $[1, -1]$ repeated 2^{m-1} times; the second half of the chain $z(h)$ is $-(z_1 \dots z_{2^m})$. Hence $z_j = z_{2^{m+1}-j+1}$ for $j = 1, \dots, 2^m$ and therefore $\tilde{g}(h) = 2 \sum_{j=1}^{2^m} z_j \phi_j < 0$, since for odd $j < 2^m$, $\phi_j < \phi_{j+1}$.

To analyze the last case, we use the fact that for a linear function, the Walsh coefficients are non-zero if and only if h is of the form 2^a for $a \geq 0$, or if $h = 0$. If $h = 1 + 2^{a_1} + \dots + 2^{a_r}$ for some $r \geq 2$, where $1 \leq a_1 < \dots < a_r = m$, then depending on the parity of r , either $z_{2^{m+1}-j+1} = z_j$ or $z_{2^{m+1}-j+1} = -z_j$: in the latter case, using the same reasoning as for the case $h_0 = 0$, we get that $\tilde{g}(h) = 0$. To analyze the former case, we observe that $\tilde{g}(h) = 2 \sum_{j=1}^{2^m} z_j \phi_j$ and then let $\tilde{h} = 2^{a_1} + \dots + 2^{a_r}$. Let $g_1(u) = 2u, 0 \leq u \leq 1$. Since $\tilde{h}$ is not of the form $\tilde{h} = 2^a$, we have that $\tilde{g}_1(\tilde{h}) = 2 \sum_{j=1}^{2^m} z_j \phi_j = 0$ and thus $\tilde{g}(h) = 0$. This concludes the proof of the lemma. $\square$

Let $\mathcal{H}$ be the set of positive integers h of the form $h = 2^a + 1$ for some $a \geq 1$. Since $\tilde{g}(\mathbf{h}) = \tilde{g}(h_1)\tilde{g}(h_2)$ the above claim also shows that $\tilde{g}(\mathbf{h}) \neq 0$ only if h_1 and h_2 are both in $\mathcal{H}$.

Lemma 2. If $k \geq 6$, if both h_1 and h_2 are in $\mathcal{H}$, then $\mathbf{h}$ cannot be in $P_n^\top$.

Proof. Suppose that $h_1, h_2 \in \mathcal{H}$. In what follows $\mathbf{C}[i, j]$ refers to the entry in position $[i, j]$ of the matrix $\mathbf{C}$.

Case 1: assume $h_1, h_2 < 2^k$, with $h_1 = 1 + 2^a$ and $h_2 = 1 + 2^b$, and $1 \leq a, b \leq k-1$. Note that $\mathbf{C}_2[1, 2] = 1$ (otherwise this would contradict the assumption that P_n comes from a $(0, 2)$ -sequence, because the first 4 points would not be a $(0, 2, 2)$ -net.) Furthermore, for $k \geq 4$, we cannot have $\mathbf{C}_2[1, 3] = \mathbf{C}_2[1, 4] = 0$, as otherwise the first 16 points would not be a $(0, 4, 2)$ -net (the first two rows of $\mathbf{C}_1$ and the first row of $\mathbf{C}_2$, restricted to size 4, would not be independent).

In order to have $\mathbf{h} \in P_n^\top$, we must have $\mathbf{C}_2^\top \tilde{h}_2 = \tilde{h}_1$ since $\mathbf{C}_1$ is the identity matrix, where $\tilde{h}$ here again is the k -bit vector representing h (so for example, h_1 has exactly two non-zero entries, one in the first position and one in position $a+1$). Since $\mathbf{C}_2[1, 2] = 1$, $\mathbf{C}_2^\top \tilde{h}_2$ is of the form $(1, 1, b_3, \dots, b_k)^\top$ unless $b = 1$, in which case it is of the form $(1, 0, b_3, \dots, b_k)^\top$. Hence if $b = 1$, we must have $a > 1$ and $\mathbf{C}_2[1, i] = \mathbf{C}_2[2, i]$ for $i = 3, \dots, k$ except for $i = a+1$, where we must have $\mathbf{C}_2[1, a+1] + \mathbf{C}_2[2, a+1] = 1$. In that case, if $a \in \{k-2, k-1\}$, then the sum of the first a entries of the first two rows of $\mathbf{C}_2$ are equal to the first a entries of the first row of $\mathbf{C}_1$, violating the condition that the first 2^a points of the sequence have $t = 0$, assuming $a \geq 3$, which holds since $a \geq k-2 \geq 4$. And if $a < k-2$, then the first two rows of $\mathbf{C}_2$ along with the first $a+1$ rows of $\mathbf{C}_1$ are not linearly independent, and this number of rows is $2 + a + 1 \leq k$, which violates the assumption that P_n is a $(0, k, 2)$ -net.

If $b > 1$, then using a similar reasoning as above, this means we must have $a = 1$, and thus we must have $\mathbf{C}_2[1, i] + \mathbf{C}_2[b+1, i] = 0$ for $i = 3, \dots, k$. This implies we must have $1 < b \leq 3$, as otherwise we would have $\mathbf{C}_2[1, 3] = \mathbf{C}_2[1, 4] = 0$ (since $\mathbf{C}_2[b+1, i] = 0$ for $b+1 > i$). If $b \leq 3$ then the sum of the first and $(b+1)$ th row of $\mathbf{C}_2$ is $(1, 1, 0, \dots, 0)^\top$, which is equal to the sum of the first two rows of $\mathbf{C}_1$. This contradicts the fact that $t = 0$ since it implies the first two rows of $\mathbf{C}_1$ along with the first $(b+1)$ rows of $\mathbf{C}_2$ do not have full rank, although the number of rows is $2 + b + 1 \leq 6 \leq k$.

Case 2: either h_1 or h_2 (and possibly both) are larger than 2^k

If $h_1 > 2^k$, then as argued before we would need to have $b = 1$ in order for $\mathbf{h}$ to be in $P_n^\top$. Using a similar reasoning as in Case 1, this implies the first two rows of $\mathbf{C}_2$ and the first row of $\mathbf{C}_1$ are not linearly independent. If $h_2 > 2^k$, then we must have $a = 1$ and therefore we must have $\mathbf{C}_2[1, 3] = \mathbf{C}_2[1, 4] = 0$ in order for $\mathbf{h}$ to be in $P_n^\top$, which we have argued cannot hold. $\square$

Note: without the baker transformation, the vectors $\mathbf{h}$ that have zeroes in all of the first k positions are in the dual, and if they have a single 1 then they are such that $\tilde{g}_1(h)$ is not 0. This is why the variance is not zero in that case.

2 NUMERICAL EXPERIMENTS

2 NUMERICAL EXPERIMENTS

2.1 Experimental Setting

As explained in the main paper, we made experiments to compare the RQMC variance for lattice rules and Sobol' nets with different types of randomizations and various integrands f . We considered the following types of RQMC point sets. For each one, we give an identifier used in the text, followed in parentheses by a shorter label used in the legends of the plots, and a short description. The last five are the same as the first five, but with a tent transformation added and a "T" added to their identifier.

Lat-RS (L-RS): lattice rule with RS;
 Sob-RDS (S-RDS): Sobol' net with RDS;
 Sob-LMS-RDS (S-LMS): Sobol' net with random LMS followed by RDS;
 Sob-NUS (S-NUS): Sobol' net with NUS;
 Lat-RST: lattice rule with RS followed by tent transformation;
 Sob-RDST: Sobol' net with RDS followed by tent transformation;
 Sob-LMS-RDST: Sobol' net with random LMS followed by RDS and tent;
 Sob-NUST: Sobol' net with NUS and tent;

The generating vector $\mathbf{a}$ for the lattice rules was found using LatNet Builder (L'Ecuyer and Munger 2016; L'Ecuyer et al. 2022) with the $\mathcal{P}_\alpha$ criterion with $\alpha = 2$, product weights with weight $\gamma_j = 2/(2+j)$ for coordinate j , for $n = 2^k$ for all $k = 8, \dots, 18$, with a component-by-component construction in up to 16 dimensions. The vector $\mathbf{a}$ can be found in the Java code. The initial generating matrices for the Sobol' nets in all cases were taken from Joe and Kuo (2008). We tried all these point sets with $n = 2^k$ points for $k = 8, \dots, 18$.

We took the following functions f from Genz (1987), Goda and L'Ecuyer (2022), L'Ecuyer et al. (2023), L'Ecuyer et al. (2024), and tried them in $s = 1, 2, 4, 8, 16$ dimensions. Here, Φ denotes the standard normal cdf.

$$\text{SmoothPerB4: } f(\mathbf{u}) = -1 + \prod_{j=1}^s [1 + (1/j)(30u_j^2(1-u_j)^2 - 1)]$$

$$\text{SumUeU: } f(\mathbf{u}) = -s + \sum_{j=1}^s u_j \exp(u_j)$$

$$\text{MC2: } f(\mathbf{u}) = -1 + (s-1/2)^{-s} \prod_{j=1}^s (s-u_j)$$

$$\text{Polynomial: } f(\mathbf{u}) = -1 + \prod_{j=1}^s [1 + (u_j - 1/2)j/s]$$

$$\text{Oscillatory: } f(\mathbf{u}) = \cos\left(\sum_{j=1}^s u_j j/s\right) - \cos\left(\frac{1}{2} \sum_{j=1}^s a_j\right) \prod_{j=1}^s \frac{2 \sin(a_j/2)}{a_j}.$$

$$\text{Gaussian: } f(\mathbf{u}) = \exp\left(\sum_{j=1}^s u_j^2\right) - (1.462651745907181)^s$$

$$\text{SmoothGauss: } f(\mathbf{u}) = -\Phi(1/\sqrt{2}) + \Phi\left(1 + s^{-1/2} \sum_{j=1}^s \Phi^{-1}(u_j)\right)$$

$$\text{PieceLinGauss: } f(\mathbf{u}) = \max\left(s^{-1/2} \sum_{j=1}^s \Phi^{-1}(u_j) - \tau, 0\right) - \varphi(\tau) + \tau\Phi(-\tau)$$

AsianSeq and AsianPCA: See the text in the paper.

2.2 $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k , for all models and all s

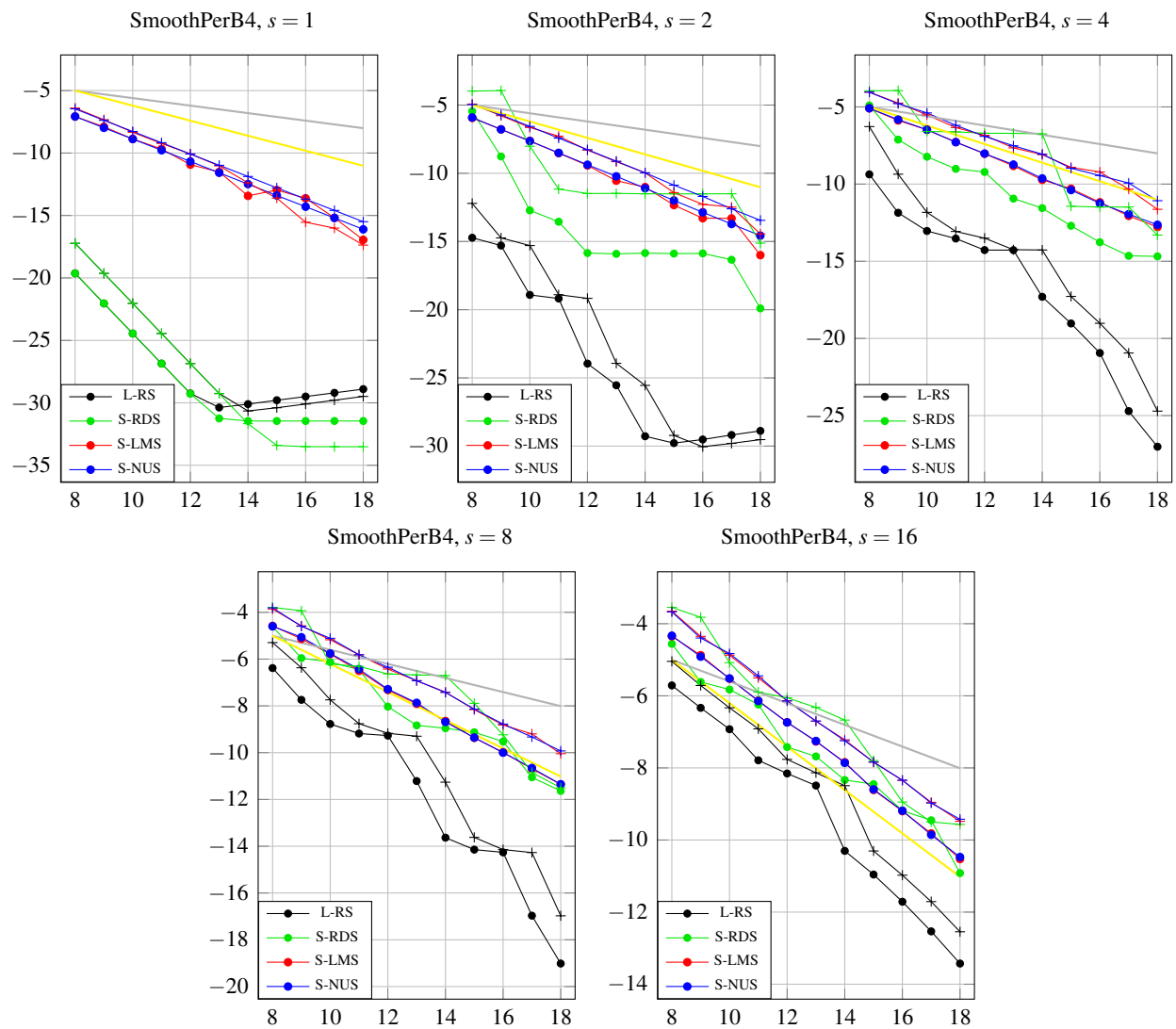
The first seven functions are all smooth, bounded, and have been recentered to have mean zero. Note that $B_4(u) = u^2(1-u)^2 - 1/30$ is the fourth Bernoulli polynomial, so `SmoothPerB4` is a weighted sum of products of Bernoulli polynomials and is one-periodic. None of the other functions is one-periodic. `SumUeU` is a sum of one-dimensional smooth functions, so very easy to integrate. `MC2` has low effective dimension since the coefficients $(s-1/2)^{-s}$ decrease rapidly with s . The `Polynomial` function is taken from Section 7.3 of L’Ecuyer et al. (2022). `Oscillatory` and `Gaussian` are from Genz (1987). `PieceLinGauss` is continuous, but not smooth and has infinite variation in the sense of Hardy and Krause. The last two functions are not in the main paper due to limited space; they have been added here. For the `AsianSeq` and `AsianPCA` functions, see the main paper.

In all cases, we pretend μ is unknown and we try to estimate it by RQMC. For each function f , each dimension s , each $k = 8, 9, \dots, 18$, and each RQMC method, we made $m = 1000$ RQMC replications with $n = 2^k$ points and computed the empirical mean and variance of the m outcomes. For $s = 1$, we took $m = 10000$ instead. All these RQMC estimators are unbiased and we want to see how their variances compare to each other and how it converges as a function of $k = \log_2 n$. The figures in Section 2.2 show the log of the variance as a function of $k = \log_2 n$ for each value of $s = 1, 2, 4, 16$, each function f , and each method. The simulations were performed with a new 64-bit version of the `hups` package, and for comparison we also show some of the results obtained with the older 32-bit version. Each plot contains straight lines to indicate the n^{-1} , n^{-2} , and sometimes the n^{-3} and n^{-4} convergence rates. The legend gives the colors used for each method without the tent transformation. When the transformation is added, we use the same color but with $\bullet$ replaced by a $+$. For validation purposes, we also plotted the empirical average as a function of k , and some of those plots are shown in Section 2.3.

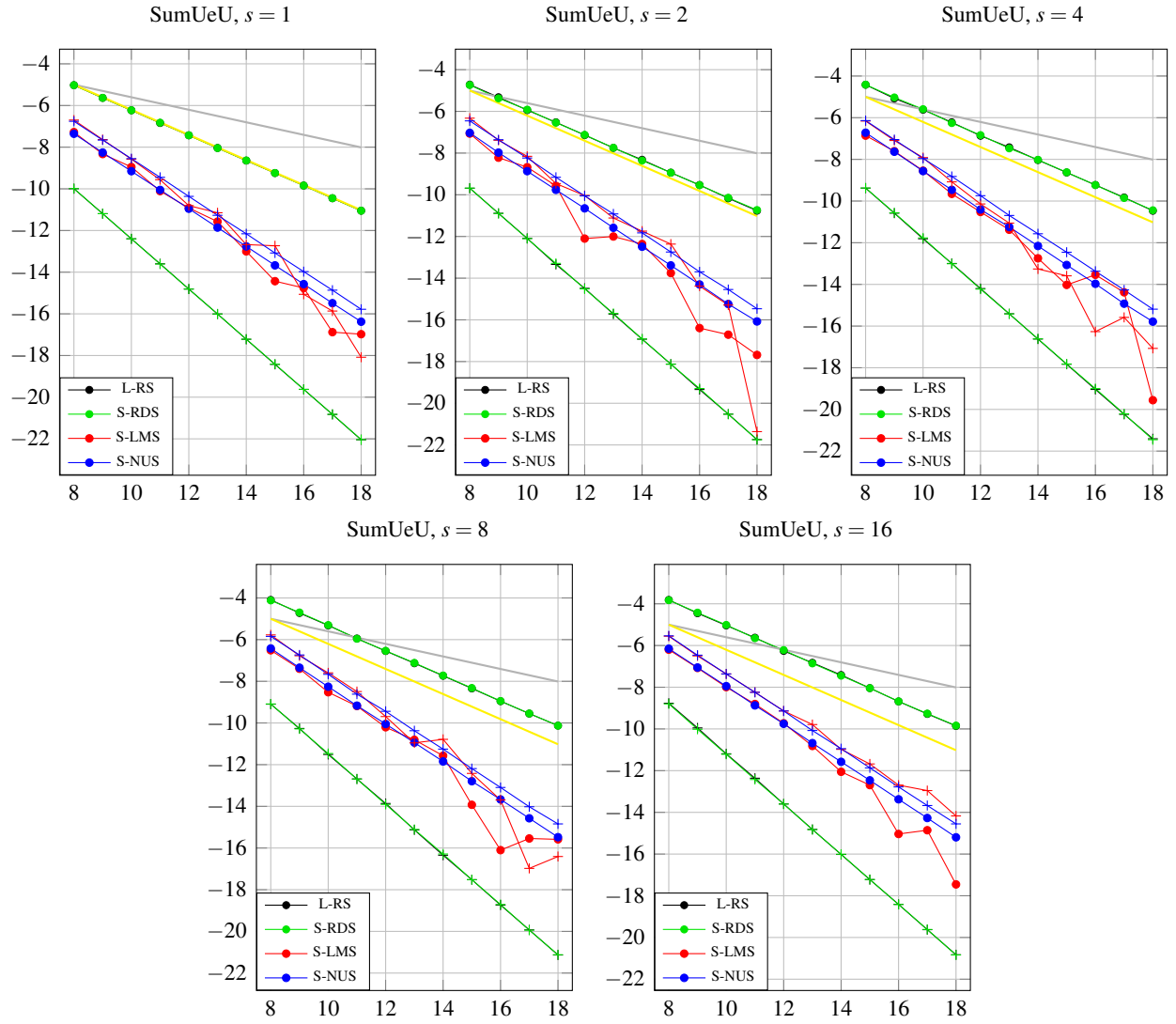
2.2 $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k , for all models and all s

The following plots show $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k for all integrands f and values of s . In those plots, the gray and yellow straight lines indicate decreases at the rates of n^{-1} and n^{-2} , respectively. The legend indicates the color for each method. The curves without tent transform use a “ $\bullet$ ” mark and those with a tent transform use a “ $+$ ” mark of the same color. The behavior of the variance plots is discussed in the main paper.

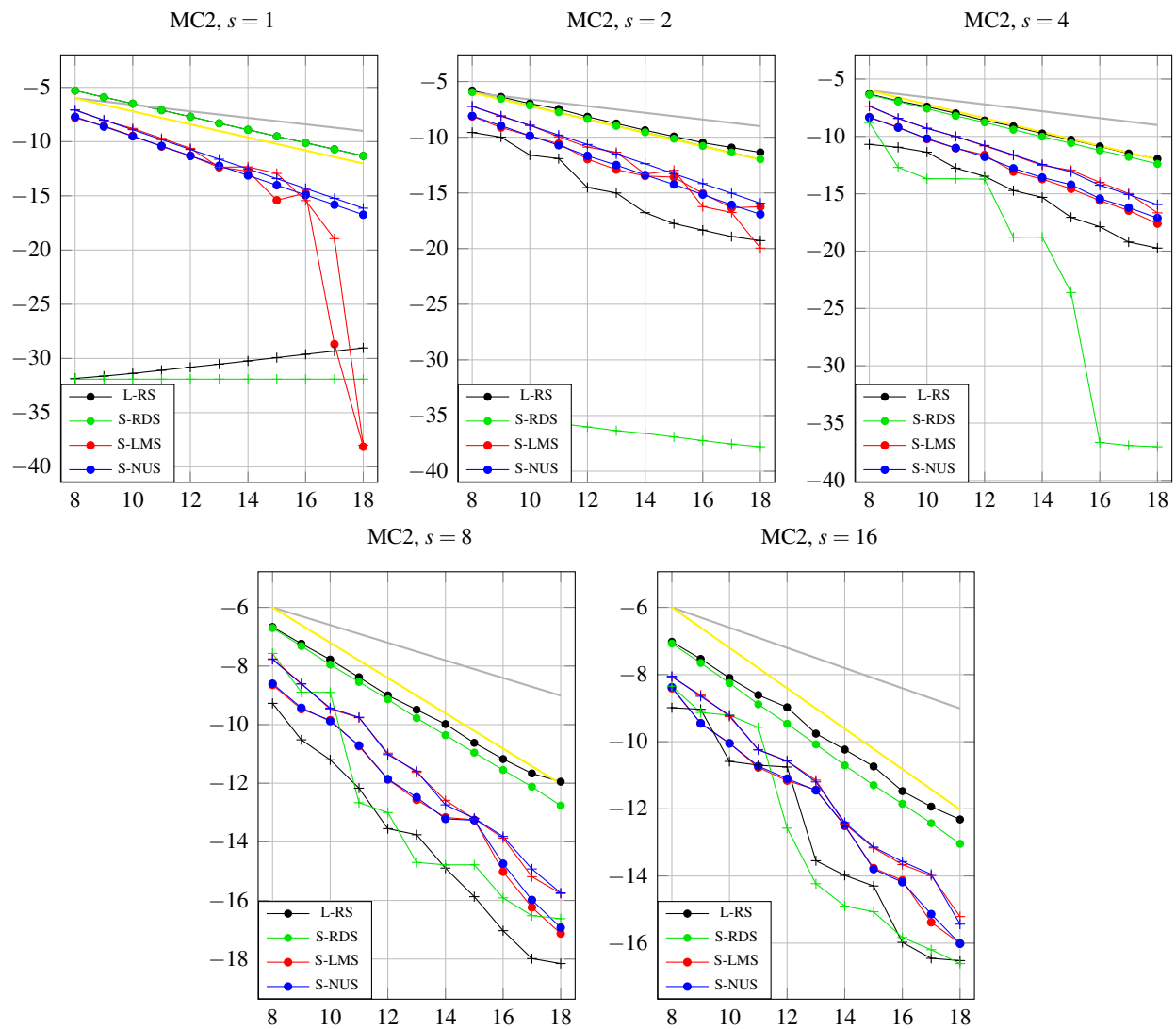
2 NUMERICAL EXPERIMENTS



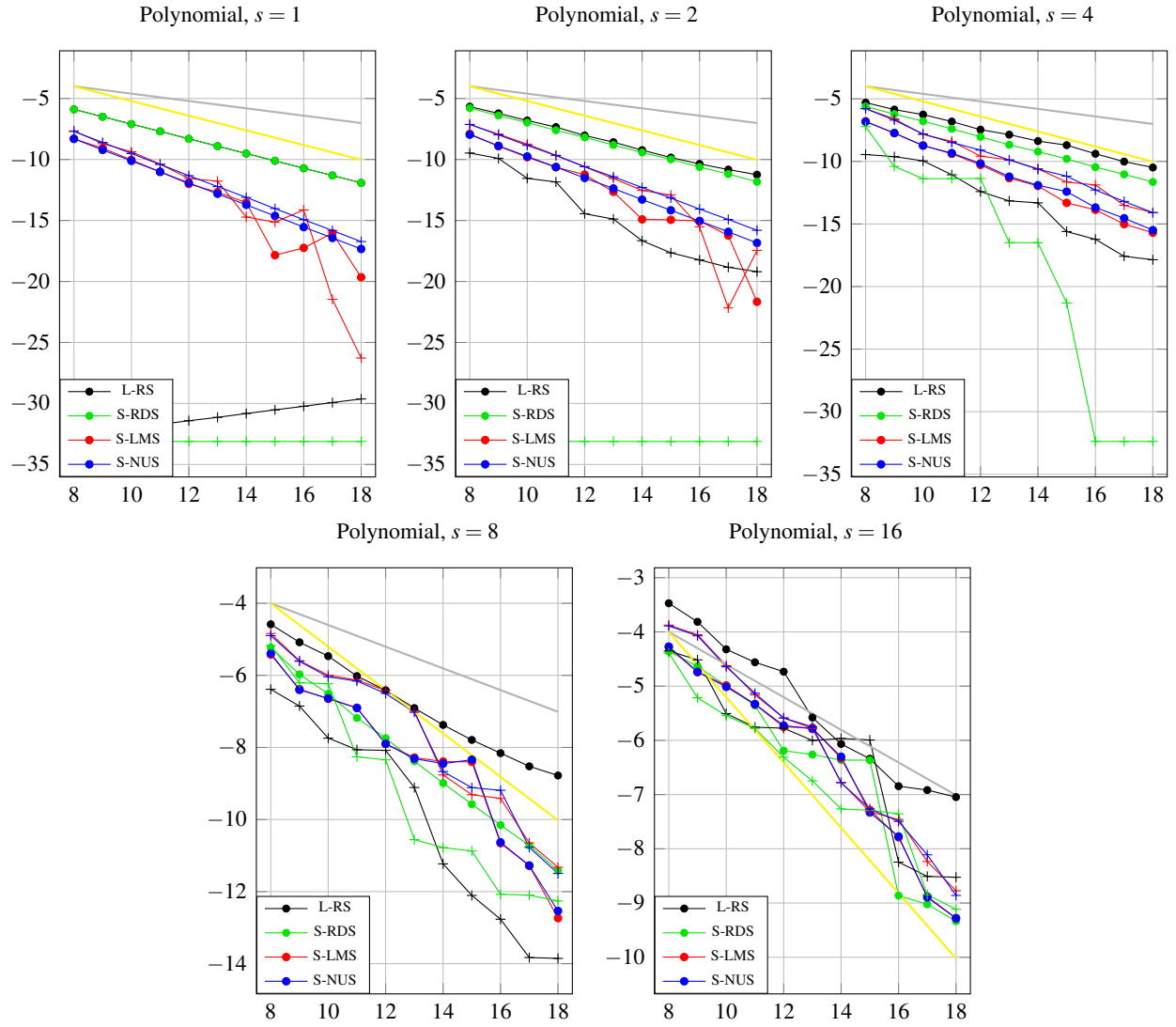
2.2 $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k , for all models and all s



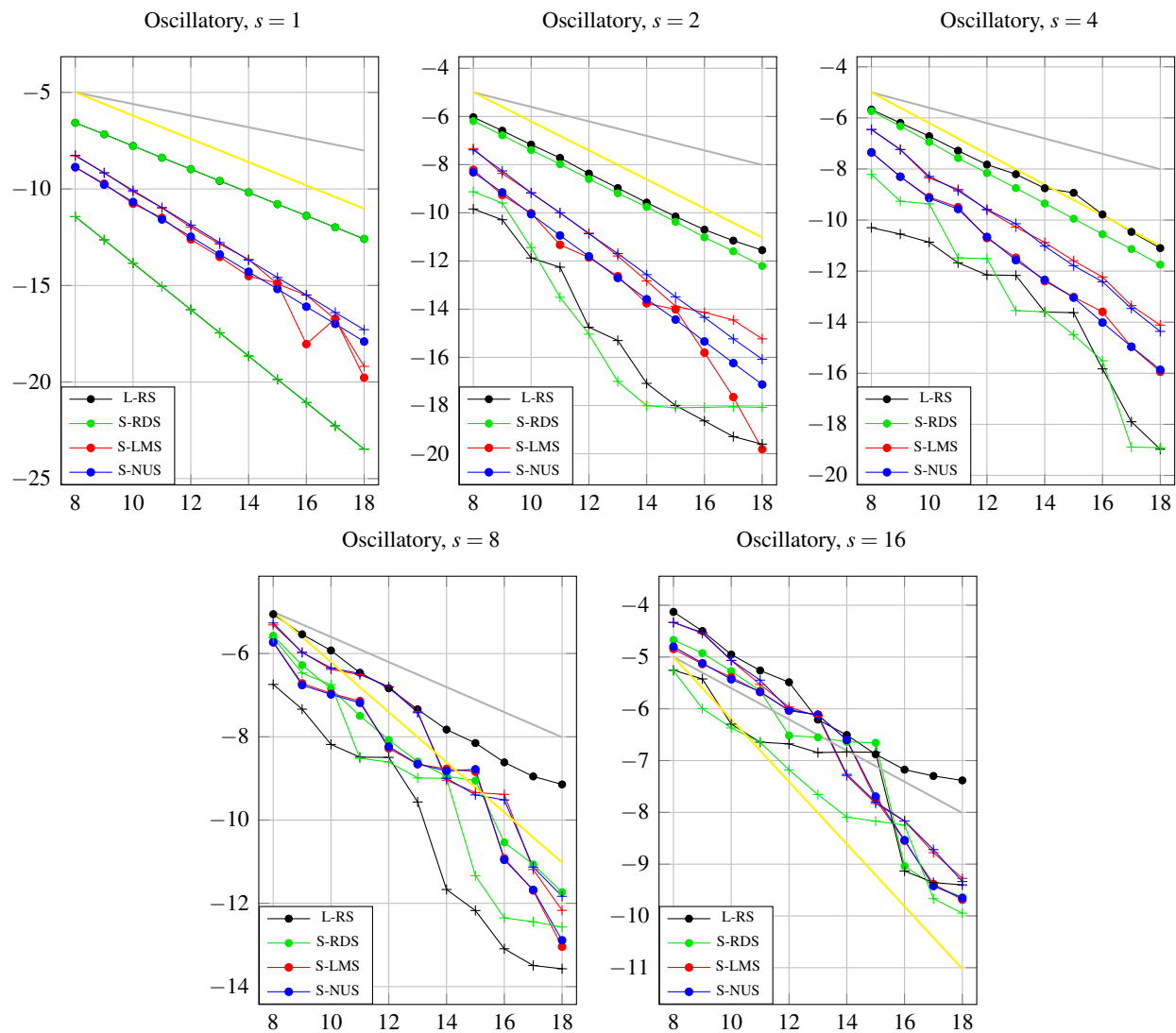
2 NUMERICAL EXPERIMENTS



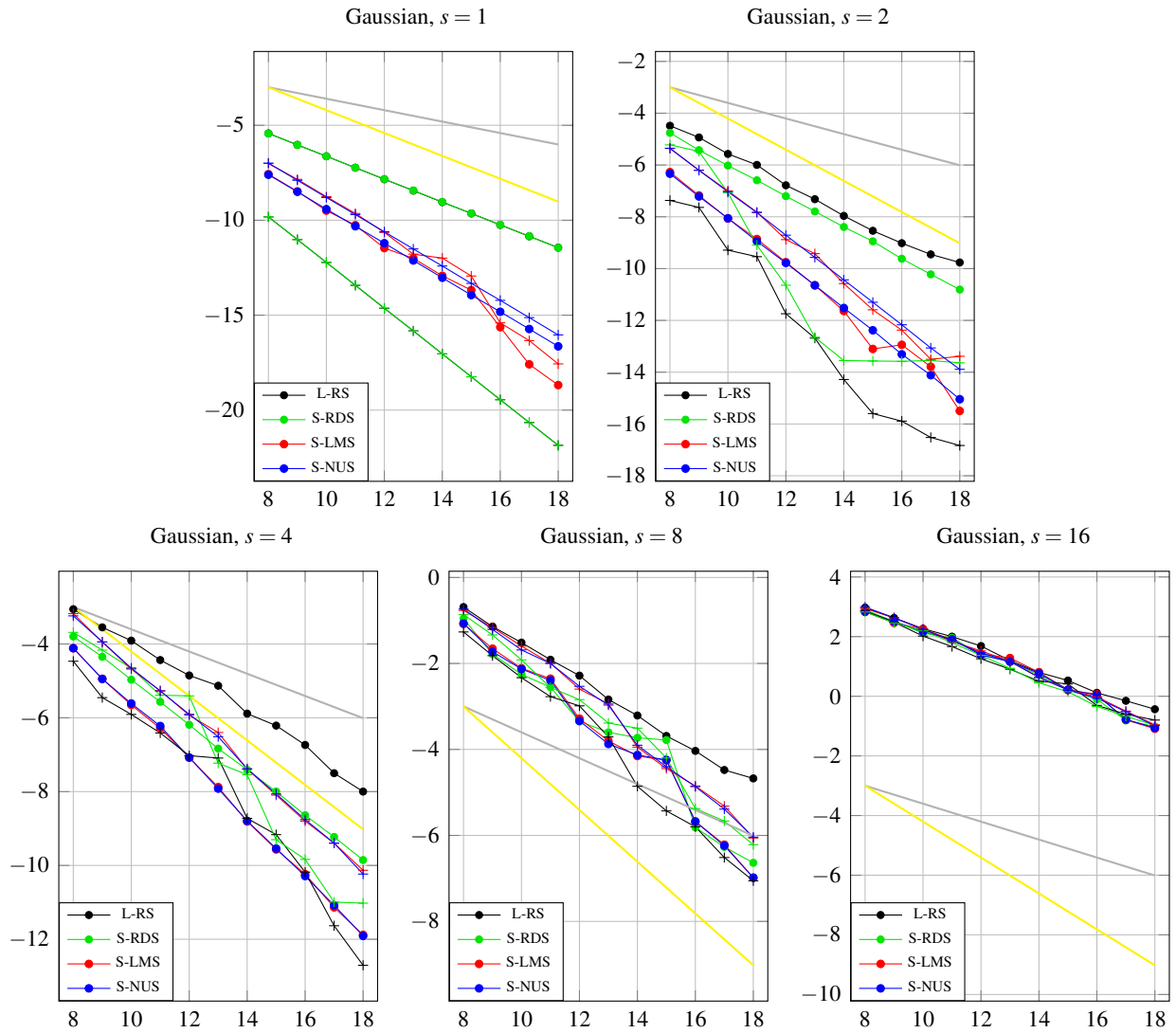
2.2 $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k , for all models and all s



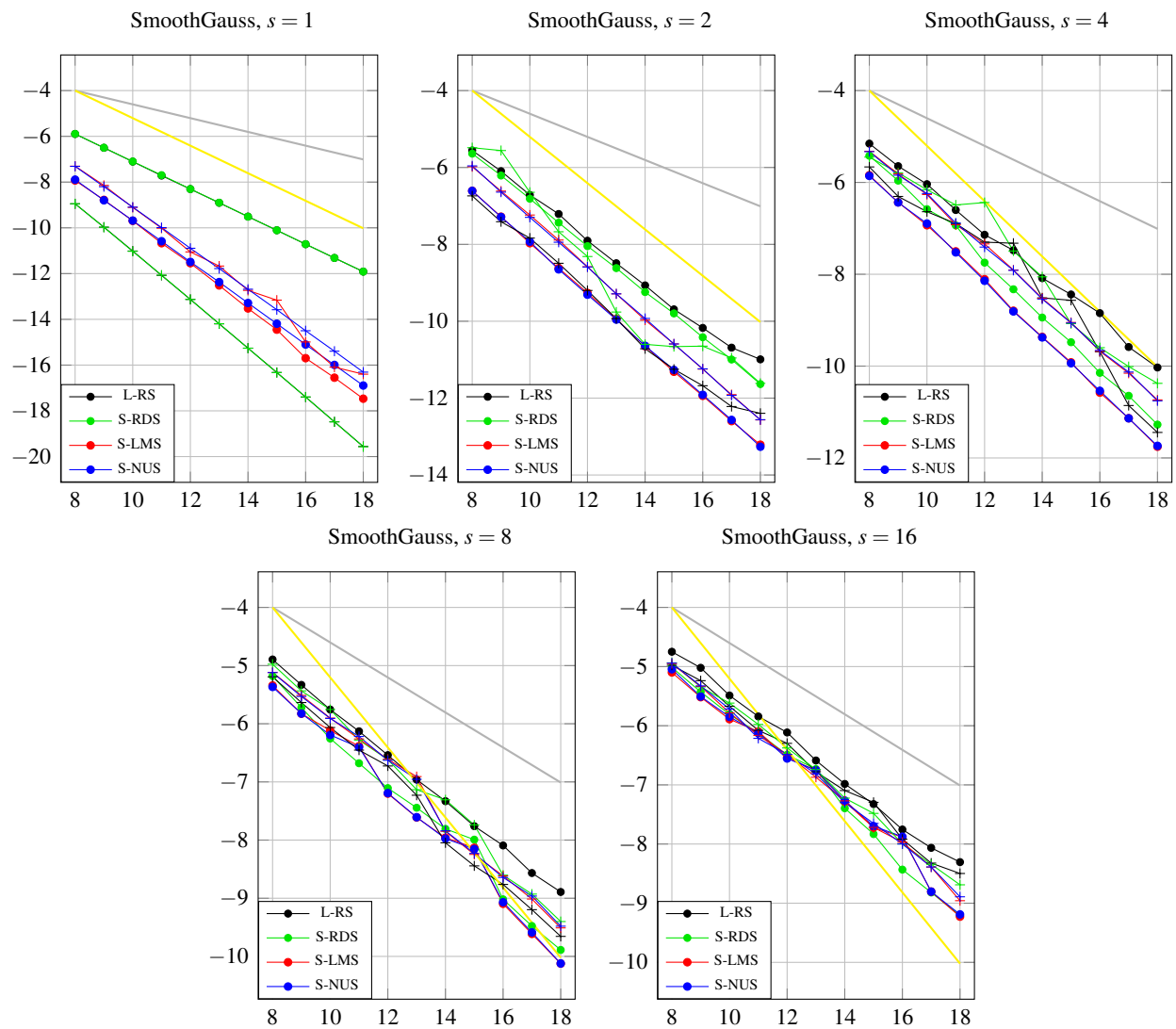
2 NUMERICAL EXPERIMENTS



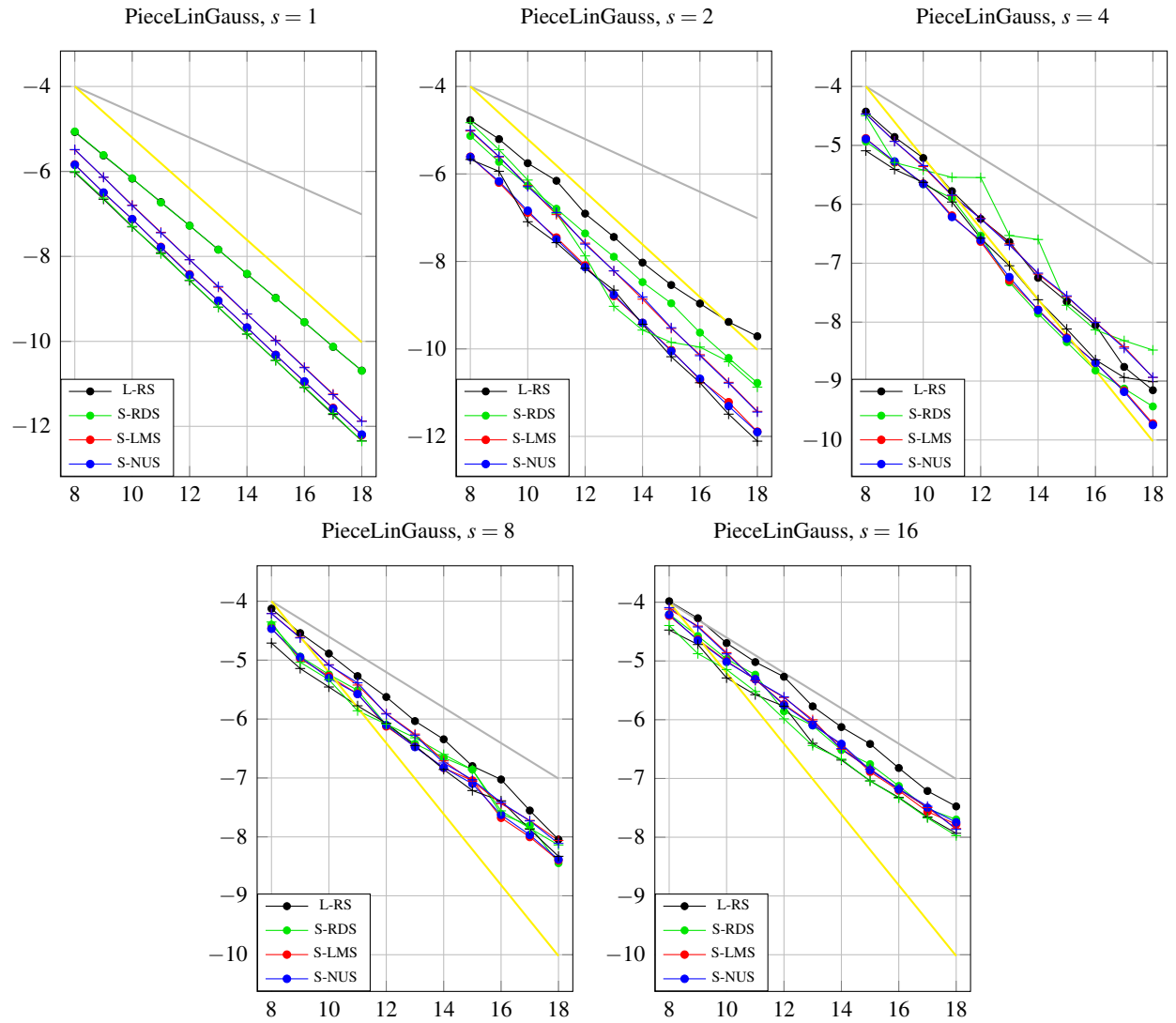
2.2 $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k , for all models and all s



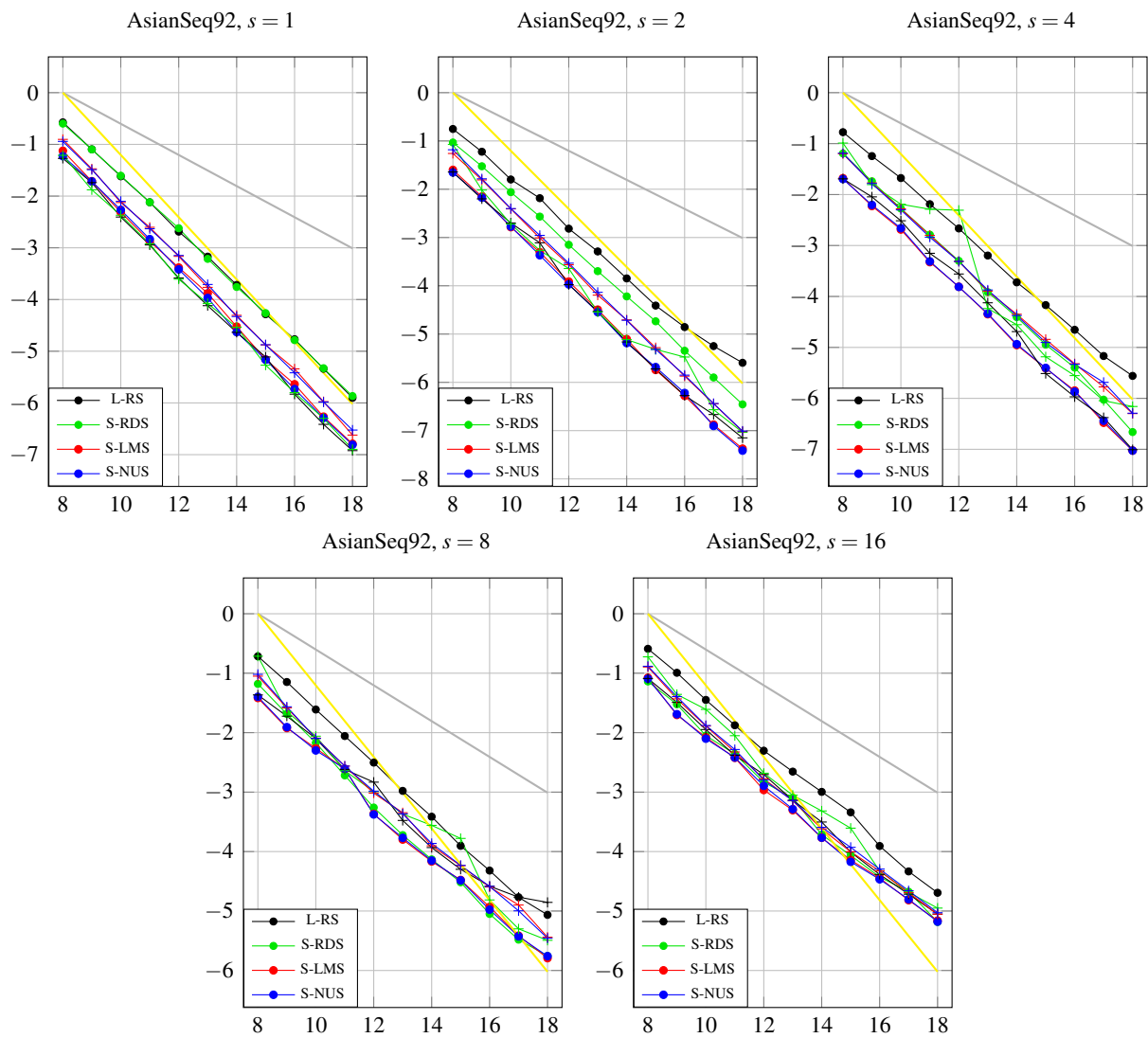
2 NUMERICAL EXPERIMENTS



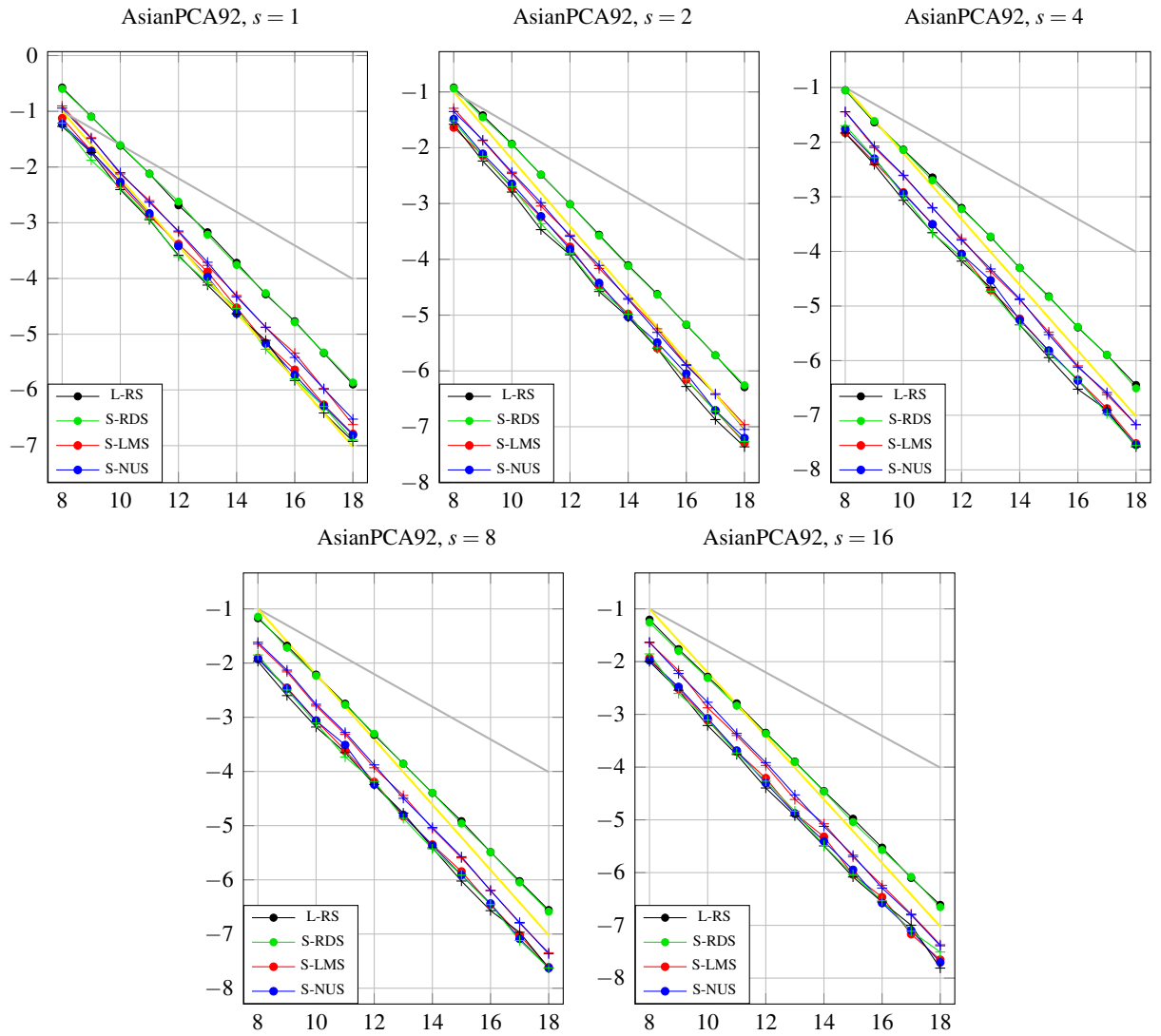
2.2 $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k , for all models and all s



2 NUMERICAL EXPERIMENTS



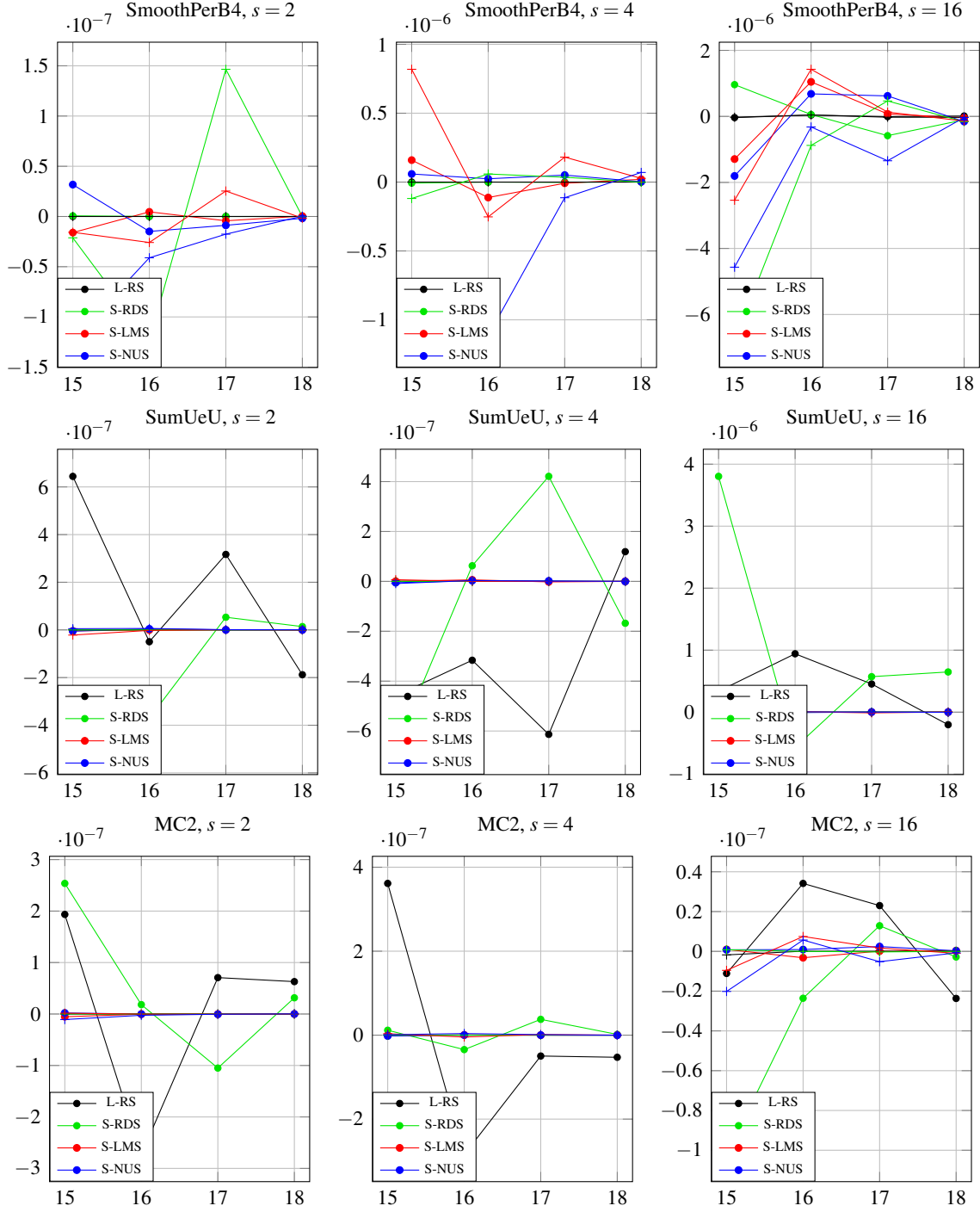
2.2 $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ as a function of k , for all models and all s



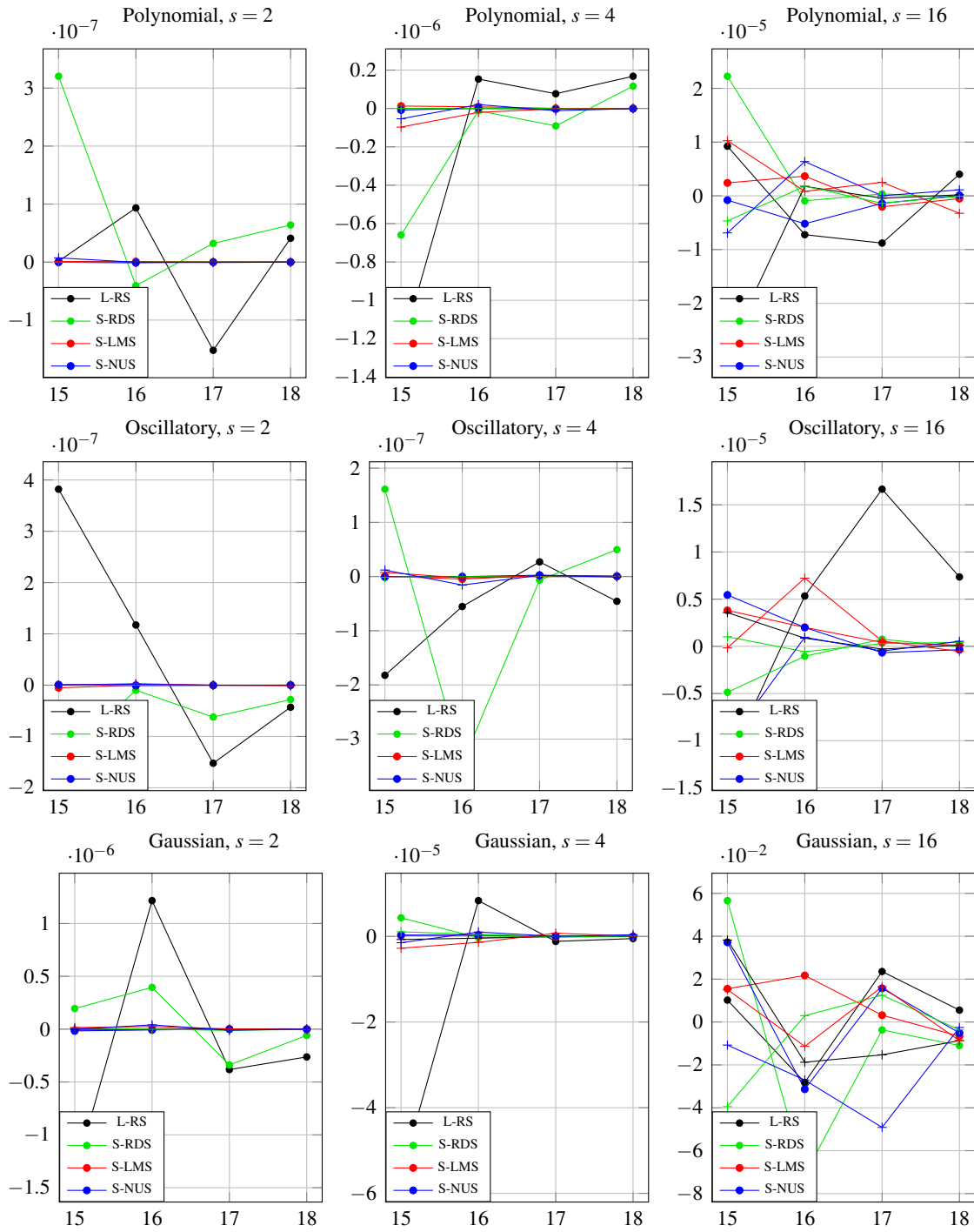
2 NUMERICAL EXPERIMENTS

2.3 Average of $\hat{\mu}_{n,\text{rqmc}}(f)$ as a function of k

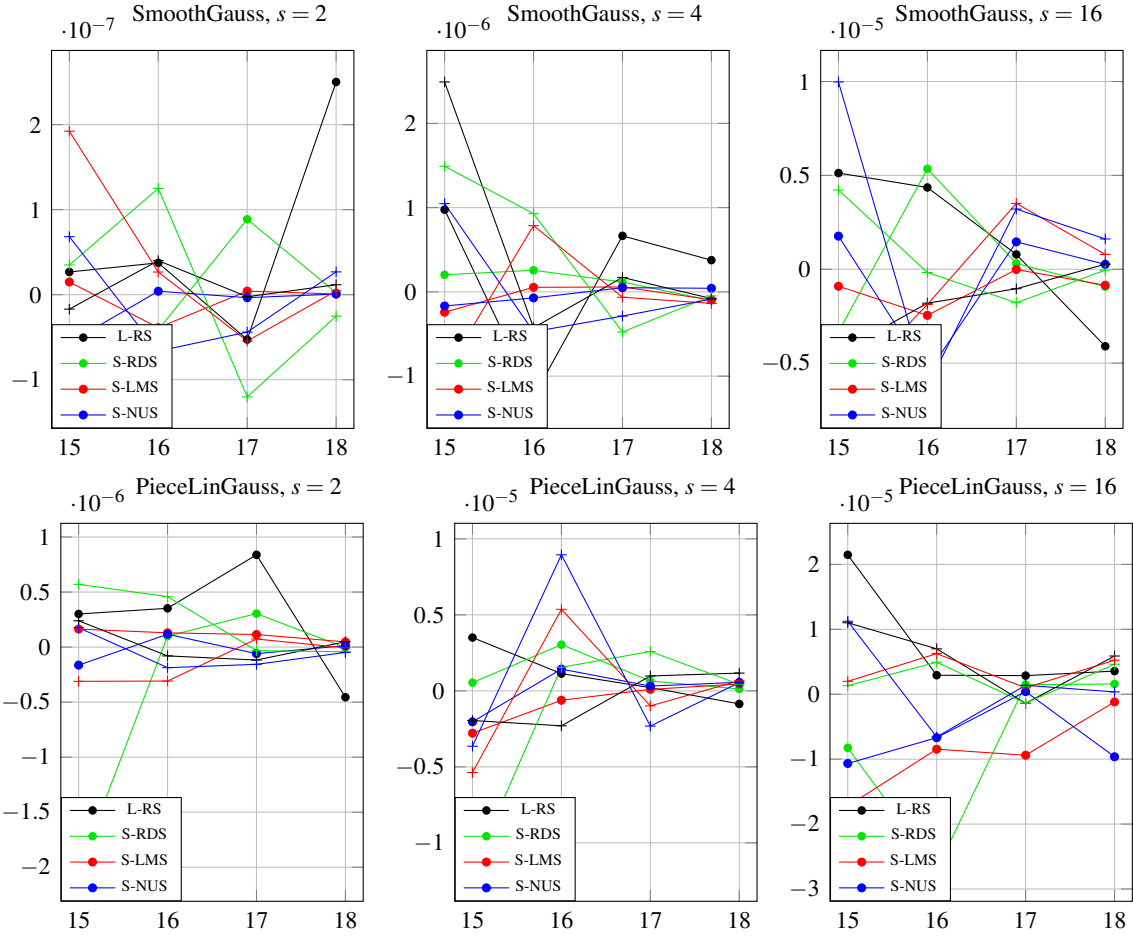
The following plots show the average value of $\hat{\mu}_{n,\text{rqmc}}(f)$ as a function of k for $k \geq 15$, for selected values of s , for the cases where the true integral is known to be zero. This was made mainly to check for correctness.



2.3 Average of $\hat{\mu}_{n,\text{rqmc}}(f)$ as a function of k



2 NUMERICAL EXPERIMENTS



2.4 Some comparisons between the scrambling and non-scrambling methods

We searched for cases where the scrambling methods Sob-NUS and Sob-LMS-RDS really give a smaller variance than the non-scrambling RQMC method Sob-RDS, Sob-RDST, Lat-RS, and Lat-RST in our results, in all 10 models, 5 values of s , and 11 values of k . Among these 550 combinations, we found 43 cases for which the empirical variance with either Sob-NUS or Sob-LMS-RDS was at least ten times smaller than with Sob-RDST, i.e., the log was smaller by at least 1. In most of those cases, one of the other non-scrambling methods Sob-RDS, Lat-RS, or Lat-RST was doing almost as well as the best scrambling method.

Table 1: $\text{Log}_{10}(\text{Var}[\hat{\mu}_{n,\text{rqmc}}(f)])$ for a few cases where the scrambling is clearly worthwhile.

model, s, k	Sob-RDS	Sob-RDST	Lat-RS	Lat-RST	Sob-NUS	Sob-LMS-RDS
SmoothPerB4, $s = 4, k = 14$	-11.56	-6.74	-17.30	-14.27	-9.63	-9.72
Oscillatory, $s = 2, k = 18$	-12.20	-18.07	-11.55	-19.60	-17.13	-19.81
Oscillatory, $s = 2, k = 18$, bis	-12.20	-18.07	-11.57	-19.59	-17.13	-17.07
Gaussian, $s = 2, k = 18$	-10.81	-13.64	-9.76	-16.83	-15.04	-15.49
Gaussian, $s = 4, k = 14$	-7.41	-7.54	-5.89	-8.72	-8.79	-8.81
AsianSeq92, $s = 4, k = 11$	-2.79	-2.29	-2.19	-3.15	-3.31	-3.33
AsianSeq92, $s = 4, k = 12$	-3.30	-2.31	-2.66	-3.56	-3.81	-3.81

2.4 Some comparisons between the scrambling and non-scrambling methods

Table 1 gives details on a few of those cases, for illustration. The first row is for SmoothPerB4 with $s = 4$ and $k = 14$. Sob-RDST performs poorly here because for that smooth periodic function, applying the tent transformation increases the variance. Note Lat-RS and Sob-RDS give much better values than the scrambling methods in this case.

For the Oscillatory function with $s = 2$ and $k = 18$ in the second row, Sob-LMS-RDS gives a much smaller empirical variance than Sob-RDST, but Sob-NUS does not. On the other hand, we know that Sob-LMS-RDS and Sob-NUS should give the same variance, which suggests that one of the variance estimate is very noisy. To check that, we repeated this case with $m = 10000$ replications. The results are in the third row. We see that the log-variance estimates are almost all the same, except for Sob-LMS-RDS, where we find -17.07 instead of -19.81. This is very close to Sob-NUS, as expected, and no better than Sob-RDST and Lat-RST. Such very noisy variance estimates often occur with Sob-LMS-RDS because this RQMC estimator often has a very high kurtosis (Pan and Owen 2024; Seljak et al. 2026).

For the two Gaussian examples in the table, both scrambling methods beat Sob-RDST, but Lat-RST does almost as well as them. This is presumably because the unscrambled Sobol' points are not so good for this function for these particular values of k . A similar behavior also appears for AsianSeq92 with $s = 4$, and is now discussed explicitly in the main paper.

In the end, we found no case where a scrambling method was really much better than all the other methods.

REFERENCES

- Genz, A. 1987. "A Package for Testing Multiple Integration Subroutines". In *Numerical Integration: Recent Developments, Software and Applications*, edited by P. Keast and G. Fairweather, 337–340. Dordrecht: Springer Netherlands.
- Goda, T., and P. L'Ecuyer. 2022. "Construction-free median quasi-Monte Carlo rules for function spaces with unspecified smoothness and general weights". *SIAM Journal on Scientific Computing* 44(4):A2765–A2788.
- Joe, S., and F. Y. Kuo. 2008. "Constructing Sobol Sequences with Better Two-Dimensional Projections". *SIAM Journal on Scientific Computing* 30(5):2635–2654.
- L'Ecuyer, P., Y. Cherkanihassani, and M. E. A. Derkaoui. 2024. "Pre-Scrambled Digital Nets for Randomized Quasi-Monte Carlo". In *Proceedings of the 2024 Winter Simulation Conference*, 443–454: IEEE Press.
- L'Ecuyer, P., P. Marion, M. Godin, and F. Puchhammer. 2022. "A Tool for Custom Construction of QMC and RQMC Point Sets". In *Monte Carlo and Quasi-Monte Carlo Methods: MCQMC 2020*, edited by A. Keller, 51–70. Berlin: Springer.
- L'Ecuyer, P., and D. Munger. 2016. "Algorithm 958: Lattice Builder: A General Software Tool for Constructing Rank-1 Lattice Rules". *ACM Transactions on Mathematical Software* 42(2):Article 15.
- L'Ecuyer, P., M. Nakayama, A. B. Owen, and B. Tuffin. 2023. "Confidence Intervals for Randomized Quasi-Monte Carlo Estimators". In *Proceedings of the 2023 Winter Simulation Conference*, 445–456: IEEE Press.
- Pan, Z., and A. B. Owen. 2024. "Super-polynomial accuracy of multidimensional randomized nets using the median of means". *Mathematics of Computation* 93(349):2265–2289.
- Seljak, G., C. Lemieux, and P. L'Ecuyer. 2026. "Using the Median vs the Average when Estimating an Expectation by Randomized Quasi-Monte Carlo". submitted for publication.